Guía de defensa del sistema de turnos médicos

Hospital Ripper · TP Desarrollo de Software · 1 de octubre de 2026

Esta guía sirve para explicar el sistema durante la defensa: qué problema resuelve, cómo circulan los datos, dónde se validan las reglas y qué evidencia demuestra el funcionamiento. El sistema permite organizar la disponibilidad de profesionales, reservar turnos y registrar la asistencia sin perder el historial.

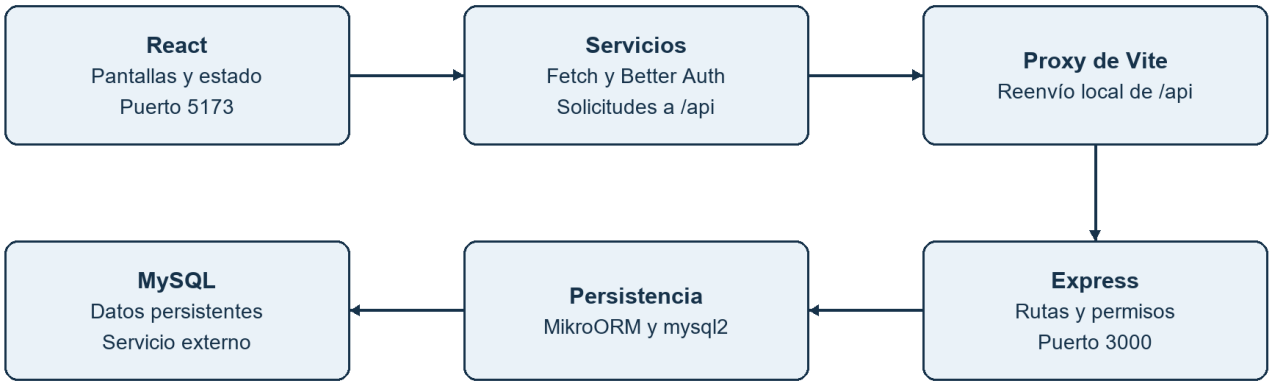


Figura 1 Recorrido general de una solicitud

La explicación central

El administrador configura los catálogos y los doctores. El doctor dispone su agenda. El paciente consulta los horarios, se registra e inicia sesión y confirma un turno. Luego el doctor utiliza esa reserva para registrar asistencia. React se ocupa de la interacción; Express comprueba identidad, permisos y reglas; MySQL conserva los datos.

Cómo usar esta guía

Primero estudiá arquitectura, modelo y autenticación. Después seguí una reserva completa y sus cambios de estado. Usá la matriz de requisitos para justificar el cumplimiento y el recorrido final para practicar la demostración. Las referencias señalan archivos y funciones concretas para abrir durante la defensa.

Estado de la revisión: hay funcionalidad y pruebas locales, pero la aprobación directa también depende de la propuesta aceptada, la documentación, el despliegue y la entrega. La propuesta enviada por privado debe conservarse junto con la aceptación del docente; trabajar individualmente requiere su autorización.

No presentes como terminado lo que figura como pendiente. La existencia de una ruta o de una pantalla no prueba por sí sola que un caso de uso esté completo.

Mapa de estudio

Orden recomendado para comprender el proyecto y explicar sus decisiones

Página	Tema	Qué deberías poder explicar
3	Requisitos técnicos	Qué evidencia cubre backend y frontend
4	Requisitos funcionales	CRUDs y casos de uso relacionados
5	Arquitectura	Capas y separación por API
6	Modelo	Entidades y relaciones
7	Autenticación	Cuenta, perfil y sesión
8	Autorización	Rol y propiedad del recurso
9	CRUDs	Catálogos, doctor y agenda
10	Disponibilidad	Cómo se generan los horarios
11	Reserva	Validaciones antes de guardar
12	Estados	Cancelación y asistencia
13	Notificaciones	Cola, envío y reintentos
14	Frontend	Estado, componentes y servicios
15	API y errores	Endpoints y códigos HTTP
16	Pruebas	Qué valida cada tipo de test
17	Configuración y pendientes	Cómo ejecutar y qué falta
18	Demostración	Recorrido para la defensa
19	Preguntas posibles	Respuestas basadas en el código
20	Fuentes	Enunciado y mapa del código

Lectura breve: páginas 3, 4, 5, 7, 11, 12 y 19. Lectura en detalle: documento completo. Las imágenes de pantallas son capturas locales del recorrido de verificación del 1 de octubre de 2026; contienen datos de demostración.

Correspondencia con los requisitos técnicos

La evidencia debe relacionar una condición del enunciado con una parte observable del proyecto

Área	Estado	Evidencia actual
Backend separado y API	Implementado	Express en app.ts y routers /api; React consume JSON.
Base externa y mapper	Implementado	MySQL como servicio; entidades y consultas con MikroORM.
Capas	Implementado	Archivos de rutas, controladores, entidades y utilidades compartidas.
Validación y errores	Implementado	Validación por recurso y manejadorErrores.ts centralizado.
Dependencias	Declaradas	package.json y lockfiles en ambos proyectos.
Pruebas de backend	Verificado	34 tests aprobados; incluyen integración con MySQL.
Login y niveles de acceso	Implementado	Better Auth; paciente, doctor y admin.
Frontend con framework	Implementado	React y TypeScript; HTML semántico, MUI y Tailwind.
Servicios y modelos	Implementado	Servicios/api.ts, servicios por recurso y Tipos/dominio.ts.
Eventos y reactividad	Implementado	useState, useEffect, props y callbacks de componentes.
Prueba de componente y E2E	Verificado	3 tests de Card; 10 pruebas Playwright.
Ambientes	Implementado	Variables .env y ramas por NODE_ENV.
SM MD LG y UX	Por completar QA	Hay estilos responsive; falta evidencia de revisión integral.

TypeScript agrega tipos y se compila a JavaScript; el backend se ejecuta con Node.js. La organización en capas existe, aunque varios controladores contienen también lógica de negocio: no conviene afirmar que hay una capa de servicios independiente para todos los recursos.

Estos estados describen la implementación y la verificación disponible. No sustituyen la aceptación académica de las tecnologías y del alcance. Fuente del criterio: enunciado de la cátedra, sección 3.1 [1].

Correspondencia con los requisitos funcionales

Una operación CRUD mantiene datos; un caso de uso combina reglas y operaciones para producir valor

Recurso	API	Interfaz y observaciones
Especialidad	CRUD	Alta, listado, modificación y eliminación administrativas.
Obra social	CRUD	Catálogo simple usado al reservar.
Motivo de cancelación	CRUD	Catálogo opcional para clasificar cancelaciones.
Doctor	CRUD con baja lógica	Depende de Usuario y se relaciona con Especialidad.
Agenda	CRUD dependiente	Pertenece a Doctor y genera horarios disponibles.
Usuario	CRUD	Registro y perfil; falta pantalla administrativa de usuarios.
Turno	Operaciones de negocio	Reserva, consulta, cancelación y asistencia; sin CRUD genérico.
Notificación	Gestión interna	No requiere pantalla CRUD para el paciente.

Casos de uso relacionados

Reservar un turno: consume agenda y doctor, comprueba disponibilidad y registra una reserva pendiente. **Registrar asistencia:** consume ese turno y registra el resultado de la atención. Son dos casos relacionados porque el primero produce el dato que necesita el segundo. Cancelar un turno agrega otra operación con valor: conserva el historial y libera el horario.

Para un integrante hay suficientes candidatos a CRUD simple y dependiente. La condición de aprobación exige al menos dos casos relacionados y cobertura de las clases de negocio; la propuesta aceptada define cómo presentar este alcance [1].

Puntos que requieren acuerdo o ajuste

El turno se cancela, no se elimina físicamente. Conviene justificar esta decisión y acordar con el docente si las operaciones de negocio cubren el alcance solicitado. Los listados administrativos permiten editar elementos, pero hay que revisar el criterio de detalle con consulta al backend [3]. No afirmar que todos los listados ya cumplen ese punto.

Código para mostrar
frontend-dsw/src/Paginas/AdminCargaDatos.tsx · línea 1
frontend-dsw/src/Paginas/AdminModificarDatos.tsx · línea 1
backend-dsw/src/recursos/turno/turno.rutas.ts · línea 1

Arquitectura y recorrido de una petición

Frontend y backend se comunican por un contrato HTTP y JSON

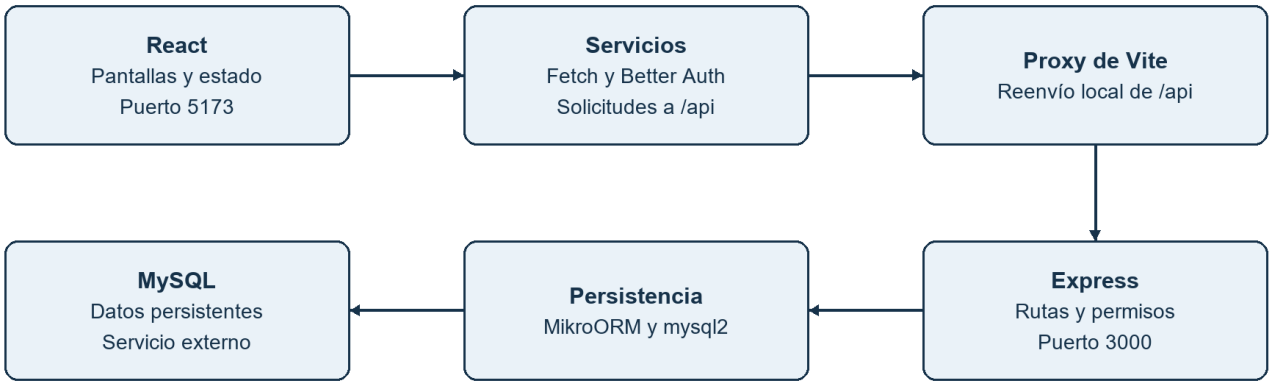


Figura 2 Dos procesos y una base externa

Desde el navegador hasta la base

Una página invoca un servicio, por ejemplo reservarTurno. El servicio utiliza api.post, que envía JSON e incluye credentials: include para que viaje la cookie de sesión. En desarrollo el navegador pide /api al puerto 5173 y el proxy de Vite reenvía la solicitud al backend en el puerto 3000.

Express identifica la ruta y ejecuta sus middlewares en orden. Primero puede comprobar sesión y rol, luego limpiar y validar la entrada y finalmente llamar al controlador. El controlador aplica las reglas del recurso y consulta o modifica entidades con MikroORM. flush materializa los cambios en MySQL y la respuesta vuelve como JSON.

Qué hace cada capa

Ruta: relaciona método y URL con permisos y función. **Controlador:** interpreta la solicitud y coordina reglas y persistencia. **Entidad:** describe propiedades, relaciones y mapeo ORM. **Servicio de frontend:** concentra llamadas HTTP. **Componente:** representa estado y atiende eventos.

RequestContext asigna un contexto de EntityManager por petición para separar el seguimiento de entidades entre solicitudes. Las tareas y transacciones crean forks. Better Auth administra sus tablas con mysql2; las entidades de negocio usan MikroORM. Ambas persistencias apuntan a MySQL.

Código para mostrar
backend-dsw/src/app.ts · línea 1
backend-dsw/src/shared/orm.ts · línea 8
frontend-dsw/src/Servicios/api.ts · línea 5
frontend-dsw/vite.config.ts · línea 1

Modelo de datos y relaciones

La identidad de acceso se vincula con el perfil utilizado por las reglas de negocio

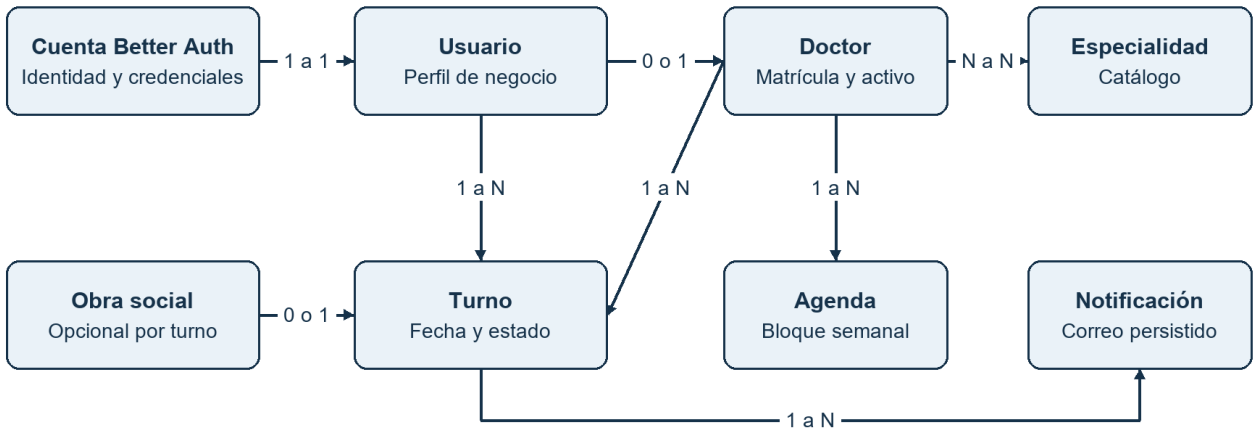


Figura 3 Relaciones principales del modelo

Usuario: tiene ID numérico, nombre, apellido, email, rol y datos de perfil. authUserId vincula el perfil con la cuenta de Better Auth. El hash de la contraseña no está en esta entidad; las credenciales están en auth_account.

Doctor: usa matrícula como clave, referencia un único Usuario y tiene activo para baja lógica. Nombre y email se leen del usuario para evitar duplicarlos. Un doctor puede tener varias especialidades y una especialidad varios doctores: relación muchos a muchos.

Agenda: cada bloque pertenece a un doctor y guarda día de semana, hora inicial, hora final y duración. Puede haber varios bloques por doctor. **Turno:** referencia paciente y doctor, conserva fecha, estado y, opcionalmente, obra social y motivo de cancelación.

Notificación: referencia el turno y guarda el trabajo de correo, su estado y reintentos. El diagrama resume el modelo; la relación opcional de Turno con MotivoCancelacion no está dibujada para mantener la legibilidad.

Decisiones que podés defender

Una agenda no es una lista de turnos ya creados: es una regla recurrente para generar posibilidades. Una reserva es un hecho concreto con fecha. La baja lógica del doctor conserva la integridad del historial. Separar cuenta y perfil permite utilizar una biblioteca de autenticación sin reemplazar las relaciones numéricas del negocio.

Código para mostrar
backend-dsw/src/recursos/usuarios/usuario.entidad.ts · línea 1
backend-dsw/src/recursos/doctor/doctor.entidad.ts · línea 1
backend-dsw/src/recursos/agenda/agenda.entidad.ts · línea 1
backend-dsw/src/recursos/turno/turno.entidad.ts · línea 1

Registro inicio y duración de la sesión

El código actual usa Better Auth con sesiones persistidas y cookies

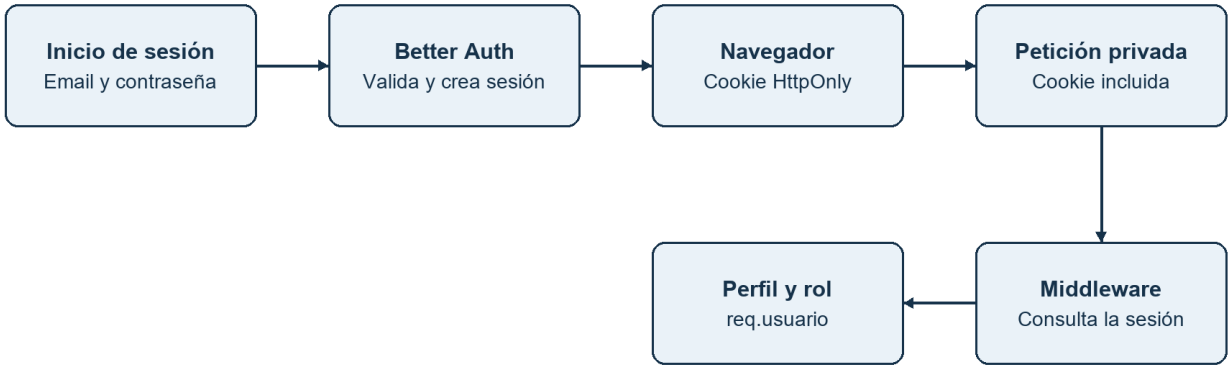


Figura 4 Autenticación y resolución del perfil

El registro público recibe nombre, apellido, DNI, email y contraseña mediante POST /api/usuarios. El controlador llama a signUpEmail. El plugin asigna paciente como rol predeterminado; el cliente no decide privilegios. Un hook valida los datos y crea el perfil Usuario vinculado a la cuenta.

autoSignIn está desactivado. Tras registrarse, la página envía al login. authClient.signIn.email valida las credenciales; la biblioteca genera la sesión y su cookie HttpOnly. Luego el frontend consulta /api/usuarios/me para obtener el perfil y mostrar nombre y menú.

La sesión dura 86.400 segundos, es decir 24 horas. disableSessionRefresh evita extenderla por actividad y cookieCache está desactivada. Al recargar, AuthContext consulta la sesión y recupera el perfil. Mientras espera, los guards muestran Cargando sesión para evitar una redirección prematura.

Al cerrar sesión, authClient.signOut invalida la sesión en el servidor y React limpia el usuario visible. No se guarda un token nuevo en localStorage; se retiran los datos antiguos. El hash de contraseña corresponde al mecanismo predeterminado scrypt de Better Auth, según la migración implementada.

Dos comportamientos distintos

Si un visitante intenta reservar o abrir una ruta privada, se lo envía a /registro. Si una solicitud recibe 401 por sesión vencida, api.ts dispara un evento y AuthContext envía a /login. Después de iniciar sesión, Login navega al inicio; el retorno automático a la pantalla previa todavía no está implementado.

Código para mostrar
backend-dsw/src/shared/auth.ts · línea 12
frontend-dsw/src/Contextos/AuthContext.tsx · línea 1
frontend-dsw/src/Servicios/authService.ts · línea 1
frontend-dsw/src/Paginas/Login.tsx · línea 1

Permisos y propiedad del recurso

Estar autenticado no implica poder modificar cualquier dato

Acción	Paciente	Doctor	Admin
Explorar especialidades y horarios	Sí	Sí	Sí
Reservar para su cuenta	Sí	Sí excepto consigo	Sí
Ver perfil propio	Sí	Sí	Sí
Editar catálogo y doctores	No	No	Sí
Administrar agenda	No	Solo propia	Cualquier doctor
Consultar turno individual	Solo propio	Propio o atendido	Sí
Cancelar turno pendiente	Solo propio	Solo como dueño	Sí
Marcar asistencia	No	Solo turno atendido	No con rol admin
Listar usuarios por API	No	No	Sí

verificarSesion consulta Better Auth y busca el perfil vinculado. Construye req.usuario con ID de negocio y rol vigente. autorizarRoles comprueba el nivel exigido por la ruta. Los controladores agregan propiedad: una agenda debe pertenecer al doctor y la asistencia al profesional que atiende el turno.

El frontend ofrece RutaPrivada, RutaDoctor y RutaAdmin y adapta el menú al rol. Es una ayuda para la navegación; la seguridad real se repite en el backend. Cambiar el rol en React o llamar la API directamente no evita la validación del servidor.

Las solicitudes que modifican recursos comprueban Origin y sec-fetch-site para rechazar orígenes ajenos. CORS permite el origen configurado con credenciales. Estas medidas complementan los controles de sesión y propiedad; CORS por sí solo no es autorización.

Ejemplo para defender

Un doctor intenta modificar la agenda de otro: tiene una sesión válida y el rol doctor, pero la propiedad no coincide. El servidor responde 403. Los tests de integración prueban este escenario. El administrador no puede marcar asistencia con su rol actual porque esas rutas exigen doctor.

Código para mostrar
backend-dsw/src/shared/middlewares/auth.middleware.ts · línea 15
backend-dsw/src/recursos/agenda/agenda.controlador.ts · línea 83
backend-dsw/src/recursos/turno/turno.rutas.ts · línea 1

Cómo funcionan los CRUDs

Los catálogos alimentan la agenda y las operaciones sobre turnos

Catálogos simples

Especialidad, ObraSocial y MotivoCancelacion tienen rutas de alta, lectura, actualización y eliminación. Las escrituras requieren admin. En el frontend se administran desde Cargar datos y Modificar datos. Los IDs autoincrementales se asignan en la base, no se ingresan manualmente para una especialidad.

Alta y baja de un doctor

El alta comprueba que no exista email ni matrícula, la longitud de contraseña y los IDs de especialidad. Crea una cuenta con rol doctor mediante Better Auth; el hook crea Usuario. Después crea Doctor, lo vincula y asigna especialidades. Si falla el guardado del doctor, intenta retirar la cuenta recién creada como compensación.

La modificación del doctor administra especialidades y activo. Sus datos personales pertenecen al Usuario. DELETE /doctores/:matricula cambia activo a false: no elimina doctor ni turnos. Los listados públicos y la disponibilidad dejan de ofrecerlo; el administrador puede verlo y reactivarlo.

Agenda dependiente

Un bloque incluye doctorId, día 0 a 6, horas HH:MM y duración positiva. Se valida la entrada y el permiso del doctor o administrador. Al editar, solo el admin puede reasignar un bloque a otro profesional. La eliminación de una agenda es física; elimina la regla de disponibilidad, no el historial de turnos.

Límite de cobertura

Usuario tiene operaciones CRUD en el backend y registro y edición de perfil en la interfaz, pero no hay una pantalla administrativa para listar y eliminar usuarios. Eliminar una cuenta asociada a doctor o turnos se bloquea para conservar relaciones y credenciales. Los turnos se gestionan con estados, no con un formulario de CRUD genérico.

Código para mostrar
backend-dsw/src/recursos/doctor/doctor.controlador.ts · línea 99
backend-dsw/src/recursos/agenda/agenda.controlador.ts · línea 83
backend-dsw/src/recursos/usuarios/usuario.controlador.ts · línea 1
backend-dsw/src/shared/auth.ts · línea 12

Cómo se calcula la disponibilidad

Los horarios se generan a partir de los bloques semanales del profesional

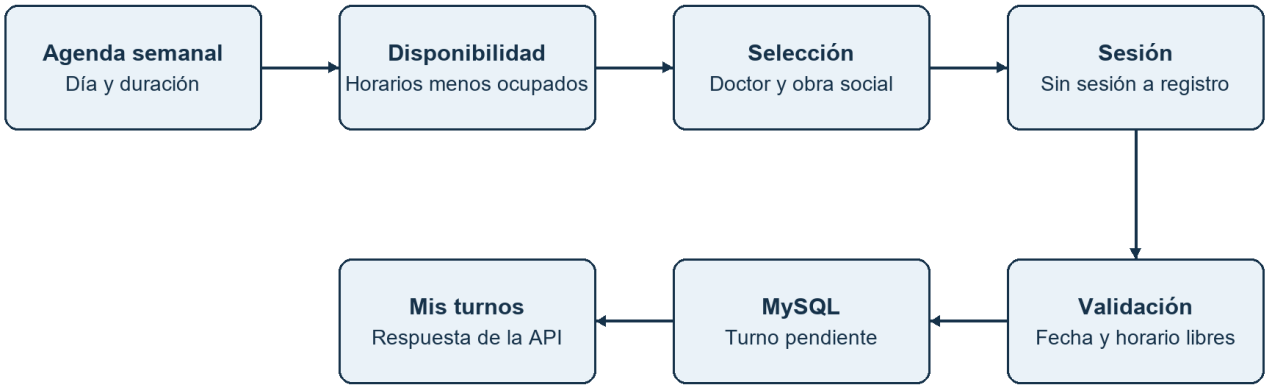


Figura 5 Relación entre agenda disponibilidad y reserva

generarSlotsDelDia obtiene el día de semana de la fecha solicitada, busca los bloques de ese doctor y genera horas desde horaInicio hasta antes de horaFin, avanzando por duracionTurnoMinutos. Con un bloque de 09:00 a 10:00 y duración de 20 minutos, produce 09:00, 09:20 y 09:40.

slotsOcupados busca los turnos pendientes del doctor en esa fecha y arma un Set de timestamps. La disponibilidad resta esos horarios a los generados. Un turno cancelado deja de ocupar el horario porque el filtro de ocupación considera únicamente pendiente.

Por especialidad se repite el cálculo para cada doctor activo y se devuelven pares de horario y profesional. El frontend agrupa por hora: primero muestra el horario y luego deja elegir entre los doctores disponibles. Por doctor se devuelve directamente su lista de horas.

Las rutas mensuales recorren los días y devuelven aquellos que tienen alguna posibilidad libre. Esa información pinta el calendario; al seleccionar el día se consulta su disponibilidad puntual. Ambas consultas son públicas, porque explorar horarios no crea una reserva.

Lo que no debés prometer

La disponibilidad actual no filtra siempre los horarios pasados al devolver la lista; el alta de la reserva sí los rechaza. El control de horarios repetidos consulta si hay un turno pendiente, pero no existe una protección atómica de reserva que garantice exclusión bajo dos solicitudes simultáneas. Es una mejora pendiente de concurrencia, no una propiedad demostrada por los tests actuales.

Código para mostrar
backend-dsw/src/recursos/turno/turno.controlador.ts · línea 302

Reserva de un turno de principio a fin

Caso de uso que combina sesión disponibilidad validación y persistencia

Desde la ficha de doctor o desde una especialidad, el visitante consulta el calendario. Al intentar reservar sin sesión se redirige a Registro con un mensaje. Después de registrar la cuenta debe iniciar sesión y volver a elegir: el código guarda from en algunas redirecciones, pero no completa la restauración de fecha y horario.

Con sesión, el usuario selecciona obra social o Sin obra social y confirma. reservarTurno envía doctorId, fechaHoraTurno y obraSocialId. El backend no recibe un usuarioId elegido por el cliente: utiliza req.usuario.id de la sesión, evitando crear la reserva a nombre de otro paciente.

Validaciones antes del guardado

Se verifica formato de fecha, que no esté en el pasado, existencia y actividad del doctor y que el usuario no reserve consigo mismo. El horario debe coincidir exactamente con uno generado por la agenda. Si ya hay un turno pendiente en ese horario, se responde 409. Una obra social enviada debe existir.

El controlador crea Turno con estado pendiente y fechaHoraEmision, hace flush y responde 201 con una representación explícita del recurso. El frontend navega a Mis turnos. La serialización incluye doctor y especialidades, sin exponer las entidades de identidad ni las credenciales.

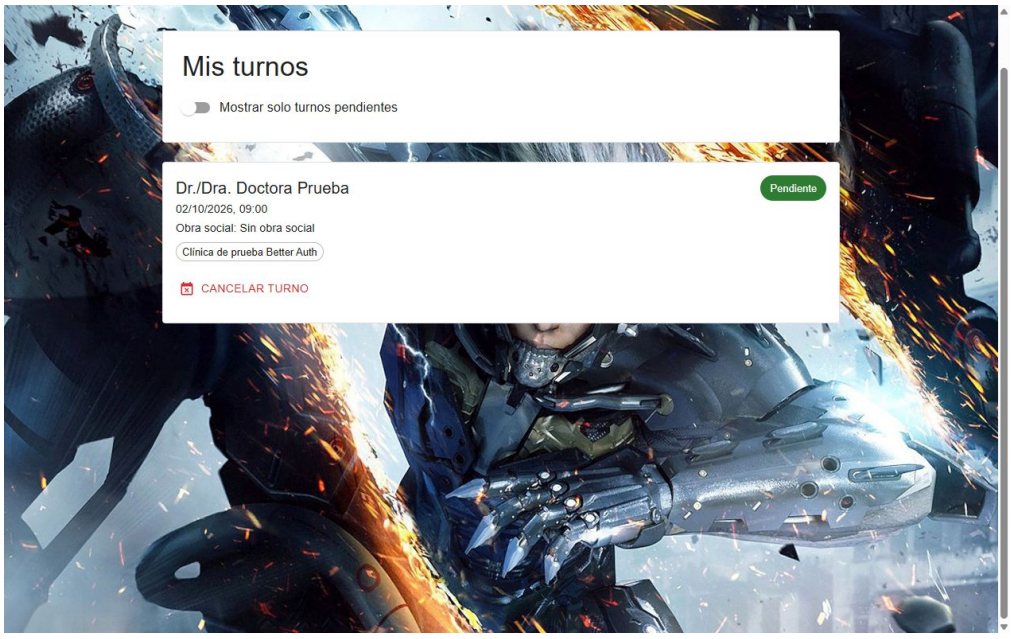


Figura 6 Reserva y sesión verificadas en el entorno local

Código para mostrar
frontend-dsw/src/Paginas/DoctorDetalle.tsx · línea 1
frontend-dsw/src/Paginas/EspecialidadTurnos.tsx · línea 1
frontend-dsw/src/Servicios/turnosService.ts · línea 1
backend-dsw/src/recursos/turno/turno.controlador.ts · línea 302

Cancelación asistencia e historial

Los cambios de estado conservan el turno y permiten relacionar los casos de uso

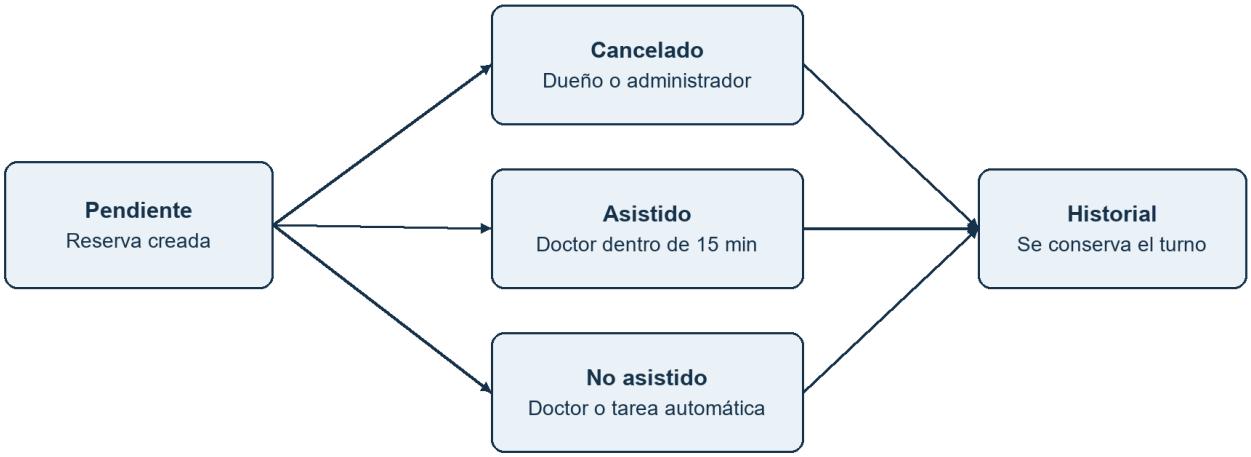


Figura 7 Transiciones de estado implementadas

Cancelación: dueño o admin, solamente sobre pendiente. Guarda estado cancelado y fechaHoraCancelacion. El texto libre y el motivo preestablecido son opcionales. Si cancela el propio dueño se encola correo de cancelación. No se borra la fila; el historial sigue mostrándola.

Asistencia: solamente el doctor del turno y mientras el turno está pendiente. El momento permitido va desde el horario del turno hasta 15 minutos después. Antes o después devuelve 409. Al confirmar, cambia a asistido y encola la notificación correspondiente.

No asistencia: el doctor puede marcar manualmente un turno pendiente. La función actual no exige esperar al horario para esa marca manual. La tarea automática busca pendientes con más de 15 minutos vencidos, los actualiza en transacciones y encola notificaciones; corre cada cinco minutos y también antes de ciertas lecturas.

Una transacción con bloqueo pesimista protege las actualizaciones de la tarea automática. No debe confundirse esta protección con la reserva: el alta de turnos no utiliza ese mismo bloqueo. Los comentarios viejos que dicen doce horas no representan la regla actual.

Qué demuestra el caso relacionado

La reserva crea el dato que luego el doctor usa para atender. Un estado final conserva el resultado del proceso. Mis turnos permite consultar historia y filtrar pendientes futuros; Turnos con pacientes permite al doctor operar sobre sus reservas. El backend tiene filtros adicionales de historia que todavía no aparecen como controles completos en esa pantalla.

Código para mostrar
backend-dsw/src/recursos/turno/turno.controlador.ts · línea 302
frontend-dsw/src/Paginas/MisTurnos.tsx · línea 1
frontend-dsw/src/Paginas/TurnosPacientes.tsx · línea 1

Notificaciones y tareas automáticas

Los correos se guardan como trabajos para poder reintentar su envío

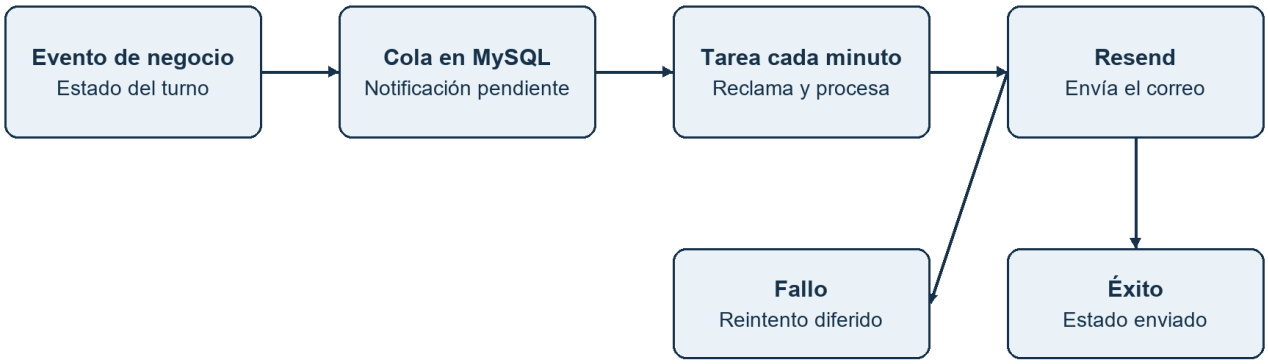


Figura 8 Flujo de notificaciones persistentes

encolarNotificacion crea una entidad con turno, tipo, destinatario, asunto, texto, estado pendiente y próximo intento. Se persiste junto al cambio de estado. La API puede confirmar el cambio sin esperar la entrega externa del correo.

iniciarNotificacionesTurnos comprueba RESEND_API_KEY y RESEND_FROM_EMAIL. Si faltan, muestra advertencia y no inicia el envío. Cuando están configuradas, ejecuta un ciclo inicial y otro cada minuto, evitando solapar ciclos dentro del mismo proceso.

El ciclo prepara recordatorios para turnos pendientes dentro de las próximas 24 horas. Usa una transacción y restricción de unicidad para evitar trabajos repetidos. Luego toma hasta 100 notificaciones vencidas y las reclama con una actualización condicional: si otra instancia la tomó, no la procesa.

La llamada a Resend usa una clave de idempotencia por turno y tipo y timeout de 15 segundos. Si funciona guarda enviado, fecha y resendId. Si falla incrementa intentos y programa una espera exponencial, limitada a 60 minutos. Un recordatorio se omite si el turno ya no está pendiente o ya pasó.

Cómo explicarlo en la defensa

La cola guarda la intención de envío incluso si el proveedor falla. Hay persistencia, reintentos y control de duplicados; no garantiza que el destinatario lea el mensaje. La implementación fue revisada, pero la entrega real de correos externos no quedó verificada en el recorrido local.

Es una funcionalidad adicional. No hay un correo inmediato de confirmación de reserva en el alta actual: existen recordatorio, cancelación, asistencia y no asistencia.

Código para mostrar
backend-dsw/src/recursos/turno/notificacionTurno.servicio.ts · línea 1
backend-dsw/src/recursos/turno/notificacionTurno.entidad.ts · línea 1

Frontend componentes estado y experiencia

React transforma acciones del usuario en cambios de estado y solicitudes al servidor

Reactividad y eventos

useState conserva formularios, errores, selección de horario, carga y diálogos. Un onChange actualiza el campo; un onClick elige o confirma; useEffect carga datos cuando cambia una dependencia. El resultado de una solicitud actualiza el estado y React vuelve a renderizar la pantalla.

Props y callbacks

Card recibe title, description y href: ejemplo de datos de entrada. CalendarioTurnos recibe fecha y fechasConTurnos; ejecuta onCambiarFecha y onCambiarMes cuando el usuario interactúa. Los formularios administrativos ejecutan onCreada u onCreado para refrescar las listas del componente padre. En React, esos callbacks cumplen el papel de comunicación del hijo al padre.

Servicios y tipos

api.ts concentra JSON, cookies y errores. Los servicios de doctor, agenda, turnos y usuario ofrecen funciones de negocio con tipos. Tipos/dominio.ts describe las formas de respuesta, como Doctor, Turno y UsuarioSesion. TypeScript ayuda durante el desarrollo, pero no valida automáticamente el JSON recibido en tiempo de ejecución.

Listado con filtro y detalle

Doctores consulta al backend por nombre y especialidad cuando cambian los filtros y conserva esos parámetros en la URL. La selección abre DoctorDetalle, que realiza una petición del doctor y muestra especialidades y calendario. Es el ejemplo más claro para demostrar listado filtrado y detalle.

Estados visibles y adaptación

Hay indicadores de carga, mensajes de error, estados vacíos y botones desactivados durante operaciones. MUI y Tailwind proporcionan reglas responsive y el menú cambia en móvil. Falta revisar y guardar evidencia integral en SM, MD y LG; usar una biblioteca no prueba por sí solo que toda pantalla sea correcta.

La foto de perfil se lee como Data URL PNG y se guarda en longtext. El frontend comprueba formato y tamaño de hasta 2 MB; no conviene afirmar una validación equivalente y completa del archivo en el backend.

Código para mostrar
frontend-dsw/src/Componentes/CalendarioTurnos.tsx · línea 1
frontend-dsw/src/Paginas/doctores.tsx · línea 1
frontend-dsw/src/Paginas/Perfil.tsx · línea 1
frontend-dsw/src/Servicios/api.ts · línea 5

API validación y manejo de errores

Las rutas representan recursos y operaciones; el servidor decide el resultado

Método y ruta	Finalidad	Acceso
POST /api/usuarios	Registro de paciente	Público
POST /api/auth/sign-in/email	Iniciar sesión	Público
GET /api/usuarios/me	Perfil de la sesión	Autenticado
GET /api/doctores/:matricula	Ficha del doctor	Público con reglas de actividad
POST /api/doctores	Crear cuenta y doctor	Admin
POST /api/agendas	Crear bloque semanal	Admin o doctor propio
GET /api/turnos/disponibilidad	Horarios por doctor y fecha	Público
POST /api/turnos	Reservar para la sesión	Autenticado
GET /api/turnos/mios	Historia del usuario	Autenticado
GET /api/turnos/atiendo	Reservas del profesional	Doctor
PATCH /api/turnos/:id/cancelar	Cancelar pendiente	Dueño o admin
PATCH /api/turnos/:id/asistio	Confirmar asistencia	Doctor del turno

La respuesta de negocio suele ser { message, data }. Los endpoints de Better Auth tienen su propio contrato. limpiarInput conserva campos permitidos; validarCamposRequeridos exige obligatorios y parsearIdNumerico rechaza IDs no numéricos. Cada recurso agrega validaciones específicas.

400: entrada inválida. **401:** falta sesión o está vencida. **403:** rol, propiedad u origen no permitido. **404:** recurso inexistente. **409:** conflicto, por ejemplo horario ocupado o transición no permitida. **500:** error inesperado con mensaje general.

Express 5 propaga los rechazos de funciones async al manejador de errores. Este traduce errores propios, de MikroORM y de Better Auth. El frontend interpreta message y muestra un Alert. El body JSON se limita a 5 MB para permitir la foto en base64; este límite no reemplaza validación del contenido.

Código para mostrar
backend-dsw/src/shared/validacion.ts · línea 1
backend-dsw/src/shared/manejadorErrores.ts · línea 1
backend-dsw/src/app.ts · línea 1

Pruebas y evidencia de funcionamiento

Cada nivel de prueba responde una pregunta distinta

Tipo	Herramientas	Evidencia de esta revisión
Unitario backend	Vitest y mocks	Validación, errores, sesión y autorización.
Integración backend	Vitest y Supertest con MySQL	Rutas, controladores, ORM, permisos y estados.
Componente frontend	Vitest y Testing Library	Card renderiza props y construye su enlace.
Navegador E2E	Playwright con API interceptada	Login, recarga, validación y redirecciones.
Recorrido real	Navegador y base local	Alta, agenda, registro, reserva y cancelación.

La última ejecución de backend aprobó 34 tests en cinco archivos y compiló TypeScript. Frontend aprobó tres tests de componente y diez de Playwright. Ambos builds pasaron; Vite informa un bundle grande como advertencia, sin impedir compilar. Estos resultados corresponden al estado local revisado el 1 de octubre de 2026.

Qué se prueba con integración

Las suites verifican alta del doctor con cuenta, permisos sobre agendas, disponibilidad pública, reserva de paciente, consulta del doctor y reglas de asistencia. También comprueban que el registro público no eleve el rol, que un cambio de rol se aplique a una sesión existente y que sesiones vencidas o cerradas ya no permitan acceder.

Cómo ejecutar las pruebas

backend-dsw: pnpm test
frontend-dsw: pnpm test
frontend-dsw: pnpm test:e2e
En ambos proyectos: pnpm build

Los E2E interceptan la API para controlar los escenarios: prueban navegación e interacción, no la integración completa con MySQL. Las pruebas de integración sí usan la base real y crean datos temporales. No existe aquí una prueba que demuestre exclusión de doble reserva simultánea ni entrega externa de correo.

Para entregar, guardar los resultados actuales, identificar qué prueba corresponde al integrante y vincular los informes desde docs/README.md. Las capturas y los informes anteriores sirven como evidencia complementaria, no reemplazan las ejecuciones actuales.

Código para mostrar
backend-dsw/src/recursos/turno/turno.estados.integration.test.ts · línea 1
frontend-dsw/src/Componentes/Card.test.tsx · línea 1
frontend-dsw/e2e/registro-rutas-privadas.spec.ts · línea 1
verificacion/better-auth.md · línea 1

Configuración ejecución y trabajo pendiente

El entorno local está preparado pero todavía no equivale a un despliegue público

Instalar dependencias en backend-dsw y frontend-dsw con pnpm install. Configurar .env a partir de cada .env.example. MySQL debe estar activo y la base configurada. En backend, DB_HOST, DB_PORT, DB_NAME, DB_USER y DB_PASSWORD conectan la persistencia; BETTER_AUTH_SECRET y APP_ORIGIN configuran autenticación.

Backend: pnpm db:prepare y luego pnpm dev
Frontend: pnpm dev
Navegador local: http://localhost:5173

db:prepare prepara el esquema de una base compatible con la migración actual. db:reset es destructivo: elimina datos y no debe usarse para iniciar una demostración. Ambos comandos de preparación actuales rechazan NODE_ENV=production. El admin se crea por un comando local auth:admin con variables ADMIN_*; no hay endpoint público de creación inicial.

En producción no se actualiza el esquema automáticamente al arrancar. Hace falta definir el procedimiento de creación y migración, el reverse proxy de /api o una configuración equivalente, HTTPS, la base accesible y las variables del entorno. No alternar localhost y 127.0.0.1 en desarrollo: son orígenes distintos para las cookies.

Pendiente	Qué hacer antes de entregar
Documentación	README principal, docs/README.md, modelo y propuesta aceptada.
API	Referencia de endpoints con ejemplos y permisos.
Gestión	Minutas, metodología y seguimiento del trabajo individual.
Entrega	Video, PR, evidencias, URL pública y cuentas de demo.
Funcionalidad	Pantalla administrativa de usuarios y revisión del detalle de listados.
QA y robustez	Tres breakpoints; revisar concurrencia de reservas y validación de datos.
Recuperación de contraseña	Implementar envío real o retirar la promesa visible si queda fuera de alcance.

El documento requerimientos-funcionales.txt describe una versión anterior: JWT, bcrypt, sanatorios y motivos obligatorios no representan el código actual. Debe actualizarse. Las condiciones de entrega se encuentran en [1] y la estructura documental en [2].

Código para mostrar
backend-dsw/.env.example · línea 1
frontend-dsw/.env.example · línea 1
backend-dsw/src/shared/preparar-db.ts · línea 1

Recorrido para practicar la defensa

Una demostración breve debe mostrar el valor del negocio y un rechazo de permisos

1. Presentar el problema: organizar disponibilidad y reservas con historial. Mostrar la separación frontend, API y base. Abrir el diagrama y ubicar app.ts, rutas, controlador y entidad.

2. Entrar como administrador: mostrar catálogos y un doctor con especialidad. Explicar la baja lógica sin ejecutarla sobre datos necesarios para la demo. Usar datos de prueba, no cuentas o información personal real.

3. Entrar como doctor: crear un bloque de agenda y explicar día, horario y duración. No confundir ese bloque con una reserva. Elegir una fecha futura que coincida con el día del bloque.

4. Salir y explorar: consultar disponibilidad sin sesión. Intentar reservar y mostrar la redirección a Registro. Registrar un paciente e iniciar sesión; volver a elegir el horario porque el retorno de la selección no está implementado.

5. Reservar: elegir obra social o Sin obra social. Confirmar y recargar Mis turnos para probar persistencia y restauración de sesión. Explicar que el usuario de la reserva surge de la sesión del servidor.

6. Cancelar: mostrar cambio a Cancelado e historial; volver a consultar disponibilidad. Como alternativa, mostrar en tests la atención: la asistencia exige un horario dentro de los 15 minutos y no se puede forzar con cualquier fecha.

7. Mostrar seguridad y tests: acceder a una ruta admin desde un paciente y explicar el rechazo del backend. Enseñar salida de pnpm test y un caso de integración. No editar fechas ni permisos reales solo para hacer pasar una demo.

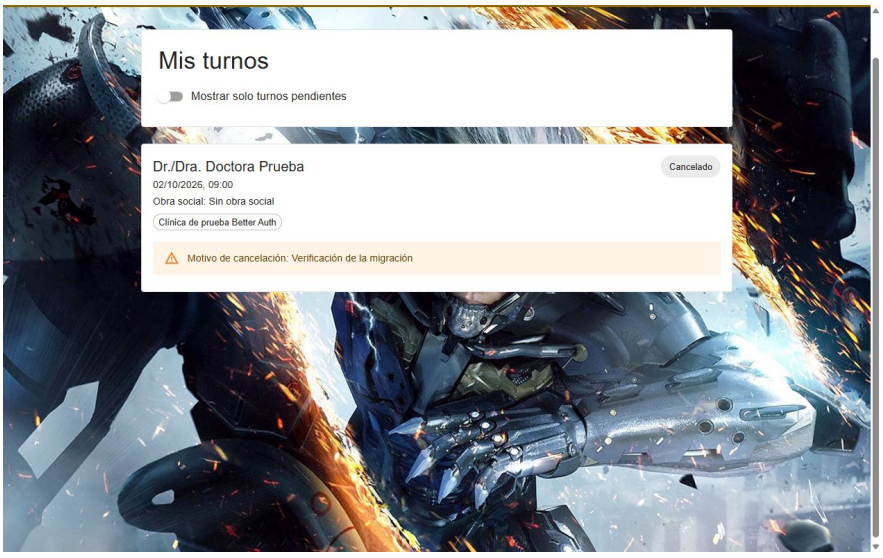


Figura 9 Cancelación conservada en el historial local

Prepará cuentas separadas y datos de demostración antes de presentar. La prueba de asistencia puede explicarse con la suite automatizada si no coincide la ventana horaria durante la defensa.

Preguntas posibles y respuestas

Respuestas breves que podés ampliar mostrando el código

Por qué usar React y Express separados

El frontend representa interacción y el backend aplica reglas y persistencia. Se comunican por HTTP y JSON. Podría cambiar la interfaz sin reemplazar el modelo de negocio si mantiene el contrato de la API.

Por qué usar un ORM

MikroORM relaciona clases y tablas, permite consultar entidades y administra sus relaciones y cambios. MySQL sigue siendo la base persistente externa. flush escribe los cambios; el ORM no elimina la necesidad de validación o de resolver concurrencia.

Autenticación y autorización significan lo mismo

Autenticación verifica quién está conectado. Autorización decide si puede hacer la operación. El doctor puede estar autenticado y recibir 403 al intentar editar una agenda ajena.

Dónde se guarda la contraseña y la sesión

Better Auth administra el hash en auth_account y las sesiones en auth_session. El navegador conserva la cookie de acceso HttpOnly. Usuario es el perfil del negocio y no almacena una contraseña propia.

Por qué no se borra el doctor ni el turno cancelado

Los turnos históricos necesitan seguir identificando al profesional. El doctor se desactiva y la cancelación cambia el estado del turno. Así se conserva la historia y se deja de ofrecer la reserva.

Qué evita una reserva inválida

El servidor comprueba sesión, profesional activo, fecha futura, pertenencia a agenda y ausencia de un pendiente en ese horario. La exclusión bajo solicitudes simultáneas todavía necesita una garantía atómica; no afirmaría que está resuelta.

Qué pasa si falla el correo

La notificación queda persistida y se reintenta con espera creciente. El resultado del turno ya está guardado. La confirmación de Resend no demuestra lectura del destinatario.

Qué falta para afirmar aprobación directa

Comparar el alcance con la propuesta aceptada, completar documentación, deploy, entrega y los puntos funcionales pendientes. Las pruebas locales demuestran escenarios concretos; no garantizan todos los requisitos ni reemplazan la evaluación del docente.

Fuentes y mapa del código

Referencias para verificar las afirmaciones y preparar la explicación oral

[1] Enunciado: github.com/utnfrrodsw/tp. Secciones 3.1, 3.2 y 3.3: requisitos técnicos, funcionales y entrega.

[2] Documentación: github.com/utnfrrodsw/tp/blob/main/docs.md. Estructura y contenido de docs.

[3] FAQ: github.com/utnfrrodsw/tp/blob/main/FAQ.md. CRUD, detalle, trabajo individual y aprobación de tecnologías.

[4] Plantilla de propuesta: github.com/utnfrrodsw/tp/blob/main/proposal.md. Conservar por separado la versión enviada y aceptada.

Para explicar	Abrir en el proyecto
Montaje de API y tareas	backend-dsw/src/app.ts
Sesión y sincronización de perfil	backend-dsw/src/shared/auth.ts
Permisos por sesión y rol	backend-dsw/src/shared/middlewares/auth.middleware.ts
MySQL y contexto ORM	backend-dsw/src/shared/orm.ts
Disponibilidad reserva y estados	backend-dsw/src/recursos/turno/turno.controlador.ts
Entrega de correos	backend-dsw/src/recursos/turno/notificacionTurno.servicio.ts
Sesión visible y cierre	frontend-dsw/src/Contextos/AuthContext.tsx
HTTP y errores de frontend	frontend-dsw/src/Servicios/api.ts
Reservas desde la pantalla	frontend-dsw/src/Paginas/DoctorDetalle.tsx y EspecialidadTurnos.tsx
Permisos y estados verificados	backend-dsw/src/recursos/turno/turno.estados.integration.test.ts
Redirecciones verificadas	frontend-dsw/e2e/registro-rutas-privadas.spec.ts

Las referencias al código y las capturas pertenecen al workspace TP-dsw-2026 revisado el 1 de octubre de 2026. Los números de línea en las páginas técnicas son ayudas para esta versión y pueden cambiar al editar. Esta guía sirve para estudiar; no reemplaza la documentación de entrega ni certifica aprobación.